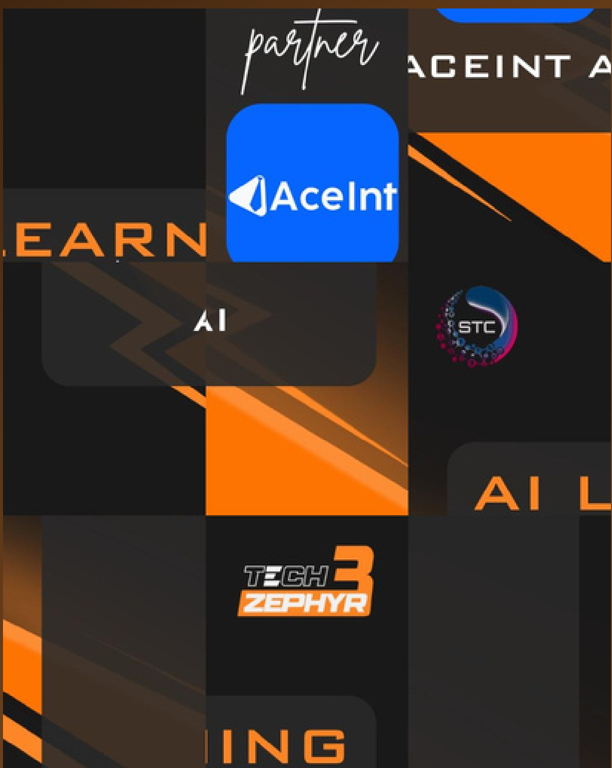
ML HACKATHON

AI JIGSAW SOLVER

AI that fits every piece!



Problem Description

This document provides the official rules for the AI Jigsaw Puzzle Reconstruction Challenge. It outlines the competition's objective, data requirements, evaluation metrics, and submission procedures. The goal of this challenge is to promote new solutions in computer vision by tackling the task of reassembling a shuffled 3x3 jigsaw puzzle. This tests a model's ability to understand visual context and spatial relationships.

Participants are tasked with developing a computational model capable of solving 3x3 jigsaw puzzles. Given a set of nine square, non-overlapping image tiles derived from an original image by dividing it into a 3x3 grid and subsequently shuffling these tiles, the model must determine the original spatial arrangement of these tiles.

Input Format

- Training Phase: Participants will use the provided training dataset (see Datasets section). The input consists of multiple puzzle sets, with each set containing 9 shuffled image tiles from one original image.
- Testing Phase: The model will be tested using a private, unseen dataset. The input format will be identical to the training data: sets of 9 shuffled tiles. Each tile will have a unique identifier (e.g., filename or index).

1	1.jpg	3 7 6 1 0 4 2 5 8

Output Format

- For each input puzzle, your model must output the correct original position for each of the 9 input tiles.
- File Format: The output must be a single CSV file or a JSON file.
- Content: Each row in the CSV will correspond to a single puzzle instance. Each row must contain a sequence of 9 integers, representing the correct tile mapping (e.g., 6,0,5,3,4,1,8,2,7).
- Representation: This sequence, P = [p_0, p_1, ..., p_8], defines the mapping from the input tile index to its original grid position.
- p_0 is the original grid position (an integer from 0-8) for the input tile at index 0.
- p_1 is the original grid position (0-8) for the input tile at index 1...and so on, up to p_8 for input tile 8.
- Grid Positions: The original 3x3 grid positions are indexed 0 through 8 (e.g., 0 for top-left, 1 for top-center, 2 for top-right, ..., 8 for bottom-right).

```json
{
  "images": [
    {
      "filename": "0.jpg",
      "sequence": [6, 0, 1, 8, 5, 3, 7, 2, 4]
    },
    {
      "filename": "1.png",
      "sequence": [0, 1, 2, 3, 4, 5, 6, 7, 8]
    }
  ]
}
```

0.jpg	6 0 1 8 5 3 7 2 4
1.jpg	3 7 6 1 0 4 2 5 8
2.jpg	5 0 2 8 7 4 1 6 3
3.jpg	6 8 4 1 3 0 7 2 5
4.jpg	7 3 1 2 8 0 5 4 6
5.jpg	8 1 0 4 5 7 3 2 6
6.jpg	5 3 8 1 7 4 2 6 0

Datasets

- Training Dataset: Participants must use the "JigsawPuzzle" dataset available on Kaggle for training:. You are expected to use this dataset exclusively for training and development.
- Test Dataset: The model's final performance will be scored on a private test dataset. This dataset contains images that are not in the training set and will not be released to the public.

Dataset link: DATASET

Judging Criteria

The performance of all submissions will be judged using the following metrics:

- Perfect Reconstruction Accuracy (PRA): This is the primary metric. It measures the percentage of test puzzles that are solved perfectly. A puzzle is considered correct only if all nine tiles are assigned to their exact original locations.
- Pairwise Adjacency Accuracy (PAA): This is a secondary metric, which may be used for tie-breaking. It measures the percentage of correctly identified "neighbor" pairs. For example, it checks if the tile predicted for the top-left position is correctly placed next to the tile predicted for the top-center position.

Submission Guidelines

- All submissions must be packaged as a single .zip file. This file must include:
- Source Code: All code files required to run your model and make predictions.
- Trained Model(s): The final model files needed by your code.
- requirements.txt: A file listing all external libraries and their specific versions (e.g., tensorflow==2.10.0).
- README.md: A guide with clear, step-by-step instructions.
- A report explaining your approach and novelty.
- Prediction Script: A main script (e.g., predict.py) that can be run to generate predictions. This script must accept a path to the test data folder and produce a single CSV or JSON file containing the predicted permutations in the format specified in Section 4.

Academic Integrity

All code and methods submitted must be the original work of the participants. While you are encouraged to use public datasets and reference existing research, plagiarism is strictly forbidden. Submitting code taken directly from public repositories or other teams is not allowed. All submissions will be checked for originality, and any violation of this policy will result in immediate disqualification.